SC6103 Distributed Systems

Course Project for 2026-2027 Trimester 1

Design and Implementation of a Distributed Flight Information System

Cross-language C++17 Server and Python Client

Team member	Matriculation number	Primary responsibility
ZHANG LIXIANG lixiang001@e.ntu.edu.sg	G2601505K	Python client
PAI LINGWEI LINGWEI001@e.ntu.edu.sg	G2608997D	C++ server

October 2026

Executive Summary

This project implements a distributed flight information system using a C++17 UDP server and a Python 3 console client. The two programs communicate exclusively through a custom language-neutral binary protocol. The implementation provides the four required services - route search, flight-detail query, seat reservation, and callback-based monitoring - plus an idempotent Set Price operation and a non-idempotent Add Seats operation.

The system implements timeout-based retransmission, simulated request and reply loss, duplicate detection, and bounded reply history. It supports both at-least-once and at-most-once invocation semantics. A deterministic reply-loss experiment shows the expected distinction: adding three seats to a flight with 40 available seats produces 46 under at-least-once because the operation executes twice, but produces the correct value of 43 under at-most-once because the duplicate receives a cached reply.

Automated cross-language tests compile and start the actual C++ server and exercise it through the Python client. All six tests pass, including variable-length string marshalling, malformed-message rejection, all service operations, callback delivery between two clients, and both invocation-semantics outcomes. The design is ready for the required multi-computer demonstration on a shared IPv4 network.

Team Members and Contributions

Member	Contact	Primary work	Contribution
ZHANG LIXIANG G2601505K	lixiang001@e.ntu.edu.sg	Python client, client-side protocol handling, console interface, retries, and callback reception	50%
PAI LINGWEI G2608997D	LINGWEI001@e.ntu.edu.sg	C++ server, service execution, flight state, monitor registry, duplicate history, and loss simulation	50%

The contribution split is recorded as 50% per member, consistent with the division between the client and server components. Both components depend on the shared protocol specification and were validated together through the integration tests.

Report Structure

- 1 Introduction and project objectives
- 2 Requirements and system architecture
- 3 Data model and binary protocol
- 4 Remote operations and error handling
- 5 Callback monitoring and concurrency
- 6 Fault tolerance and invocation semantics
- 7 Implementation details
- 8 Experiments and validation
- 9 Deployment and demonstration
- 10 Limitations and conclusion

1 Introduction

1.1 Objective

The objective is to design and implement a client-server flight information system over UDP. The project applies interprocess communication, remote invocation, explicit data representation, callbacks, timeout and retransmission, duplicate suppression, and invocation semantics in one working system. The submitted implementation also addresses the optional cross-language requirement by separating the C++ server from the Python client.

1.2 Scope

The server stores all flight information in memory for the lifetime of the process. Multiple clients may issue requests, and multiple clients may monitor the same or different flights concurrently. The client provides a repeated text menu and an explicit exit option. Every request, reply, callback, and error travels as a sequence of bytes in one UDP datagram.

Persistent storage, authentication, encryption, and graphical interfaces are outside the required scope. The implementation assumes that ordinary service requests are sufficiently separated in time for a single-threaded server event loop, as allowed by the project brief. Monitor registrations remain concurrent because the server retains multiple callback records while continuing to process requests.

1.3 Main outcomes

- A C++17 server and Python 3 client interoperate through an independently implemented binary protocol.
- All four required services and two additional operations are available through the client menu.
- The callback mechanism notifies every registered client during its monitor interval.
- At-least-once and at-most-once semantics can be selected at program start and compared under controlled message loss.
- Automated tests verify protocol behavior, services, error paths, callbacks, and non-idempotent duplicate execution.

2 Requirements and System Architecture

2.1 Deployment model

The server binds an IPv4 UDP socket to a configurable address and port, with port 8888 used by default. A client receives the server host and port as command-line arguments. The server learns each client's address and source port from recvfrom, so no client configuration is required at the server. This address is also the callback destination recorded by a monitoring registration.

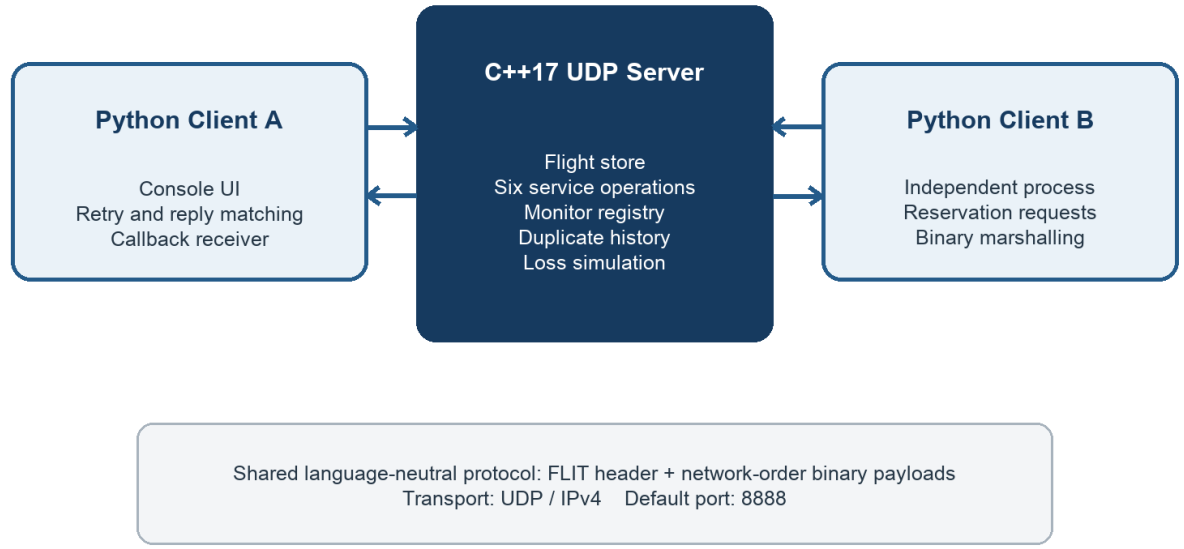


Figure 1 Cross-language UDP architecture and callback path

2.2 Component responsibilities

Component	Language	Responsibilities
Client	Python 3	Console menu, request IDs, binary marshalling, timeout and retransmission, reply matching, error presentation, and callback reception
Server	C++17	UDP receive loop, request validation, six service operations, in-memory flight state, monitor registry, callbacks, loss simulation, and duplicate history
Network protocol	Language-neutral	Fixed header, operation-specific payloads, network byte order, variable-length UTF-8 strings, and status codes

2.3 Cross-language interoperability

The client and server do not share runtime objects or serialization libraries. Python uses the struct module with explicit network-order format markers. C++ builds and reads byte vectors with explicit shifts and copies IEEE-754 double bit patterns using memcpy. Both implementations therefore depend only on the documented byte layout, not on compiler structure packing, host endianness, Python object representation, or language-specific metadata.

3 Data Model and Binary Protocol

3.1 Flight record

Field	Wire representation	Meaning
flight_id	uint32	Unique flight identifier
source	uint16 length + UTF-8 bytes	Variable-length departure place
destination	uint16 length + UTF-8 bytes	Variable-length arrival place
hour	uint8	Departure hour from 0 to 23
minute	uint8	Departure minute from 0 to 59
price	IEEE-754 float64	Non-negative airfare
seats	uint32	Number of available seats

All multi-byte numeric values use big-endian network byte order. Strings are encoded as UTF-8 after a two-byte unsigned byte length. The length describes bytes rather than characters, so non-ASCII locations remain unambiguous. The flight time is represented by separate one-byte hour and minute fields, avoiding locale-dependent date formatting.

3.2 Common message header

Every datagram begins with the same 16-byte header. The payload length is checked against the actual datagram length before any operation-specific decoding occurs.

Offset	Size	Field	Description
0	4	Magic	ASCII FLIT identifies the protocol
4	1	Version	Protocol version, currently 1
5	1	Kind	1 request, 2 reply, 3 callback
6	1	Operation	Operation code from 1 to 6
7	1	Status	Success or error category
8	4	Request ID	Reply matching and duplicate detection
12	4	Payload length	Exact number of remaining bytes

3.3 Message kinds and status values

REQUEST messages carry operation arguments. REPLY messages repeat the request ID and operation so the client can reject unrelated or stale datagrams. CALLBACK messages use the monitoring operation code and carry the flight ID and updated seat availability. Replies use the following status values: OK, BAD_REQUEST, NOT_FOUND, INSUFFICIENT_SEATS, and INTERNAL_ERROR. Error payloads contain a length-prefixed UTF-8 explanation for the user.

3.4 Defensive decoding

Both sides reject a short header, incorrect magic or version, an unsupported message kind or operation, inconsistent payload length, truncated fields, and unexpected trailing bytes. The client also checks the sender address, message kind, request ID, and operation before accepting a reply. These checks prevent an unrelated UDP datagram from being interpreted as a valid response.

4 Remote Operations

Code	Operation	Request payload	Successful result	Property
1	Search Flights	source, destination	all matching flight IDs	Idempotent
2	Get Flight	flight ID	time, price, seats, and route	Idempotent
3	Reserve Seats	flight ID, seat count	reserved and remaining seats	Non-idempotent
4	Monitor Seats	flight ID, seconds	registration ACK and callbacks	Registration refresh
5	Set Price	flight ID, absolute price	updated price	Idempotent additional operation
6	Add Seats	flight ID, seat count	added and total seats	Non-idempotent additional operation

4.1 Search and detail queries

Search Flights compares the requested source and destination with the server's in-memory records and returns every matching identifier. The default dataset includes two Singapore-to-Tokyo flights, so a successful request demonstrates a variable-length result. Get Flight returns a complete flight record. Missing routes and unknown identifiers produce NOT_FOUND rather than an empty or ambiguous success result.

4.2 Reservation

Reserve Seats validates that the requested count is positive, confirms that the flight exists, and verifies sufficient availability. On success, the server subtracts the requested count, constructs the reply, and notifies active monitors. An excessive request returns INSUFFICIENT_SEATS and does not modify the flight. Because subtraction changes state each time, reservation is non-idempotent and exposes duplicate execution under at-least-once semantics.

4.3 Additional operations

Set Price assigns an absolute non-negative airfare. Repeating the same request produces the same state, so the operation is idempotent. Add Seats increments availability by a positive amount after checking for unsigned-integer overflow. Repeating it changes state again, so it is non-idempotent. This pair gives a clear controlled comparison of how duplicate execution affects different operation classes.

4.4 Input and service errors

Condition	Status	State change
Unknown flight or no route match	NOT_FOUND	None
Reservation exceeds availability	INSUFFICIENT_SEATS	None
Zero seat count, negative or non-finite price, or invalid monitor interval	BAD_REQUEST	None
Malformed or inconsistent datagram	Rejected or BAD_REQUEST	None
Unexpected server exception	INTERNAL_ERROR	None intended

5 Callback Monitoring

5.1 Registration lifecycle

A monitor request contains a flight ID and interval in seconds. After validating that the flight exists and that the interval lies between zero and 3,600 seconds, the server stores the sender's IPv4 address, UDP source port, flight ID, and monotonic expiry time. A retransmitted registration replaces the existing record for the same client and flight instead of creating duplicate callbacks.

1. Client A sends a monitor registration for a selected flight and receives an acknowledgement containing current availability.
2. Client A blocks user input and waits on the same UDP socket until the local monitor deadline.
3. Client B reserves seats or adds seats on the monitored flight.
4. The server updates its flight state, removes expired registrations, and sends one callback to every active matching client.
5. Client A verifies the server address and callback fields, displays the new seat count, and continues waiting until the interval expires.

5.2 Concurrent monitors

The server does not need one thread per monitor. Registrations are passive records in a vector, and the event loop evaluates their expiry when processing traffic or socket timeouts. A state-changing operation iterates over active records and sends callbacks to all matching destinations. This supports multiple concurrent monitoring clients while preserving the single-threaded request-processing assumption.

6 Fault Tolerance and Invocation Semantics

6.1 Timeout and retransmission

The Python client allocates a 32-bit request ID, sends a request, and waits for a matching reply until its configurable timeout. If no acceptable reply arrives, it retransmits the identical datagram with the same request ID. The default permits four total attempts. Reusing the request ID is essential: it allows the at-most-once server to recognize that a later datagram represents the same invocation rather than a new operation.

6.2 At-least-once

In at-least-once mode, the server executes every valid request datagram it receives. Retransmission therefore ensures that an operation eventually runs when losses are temporary, but a lost reply can cause a completed operation to run again. Queries and Set Price tolerate this duplication; Reserve Seats and Add Seats do not.

6.3 At-most-once

In at-most-once mode, the server keys a bounded history by client IPv4 address, client UDP port, and request ID. Each entry stores the original request bytes and fully encoded reply. The reply enters history before transmission. If the reply is lost and the request is repeated, the server returns the cached bytes without executing the operation again. Reusing an ID with different request bytes produces BAD_REQUEST. The oldest entries are removed when the 2,048-entry limit is exceeded.

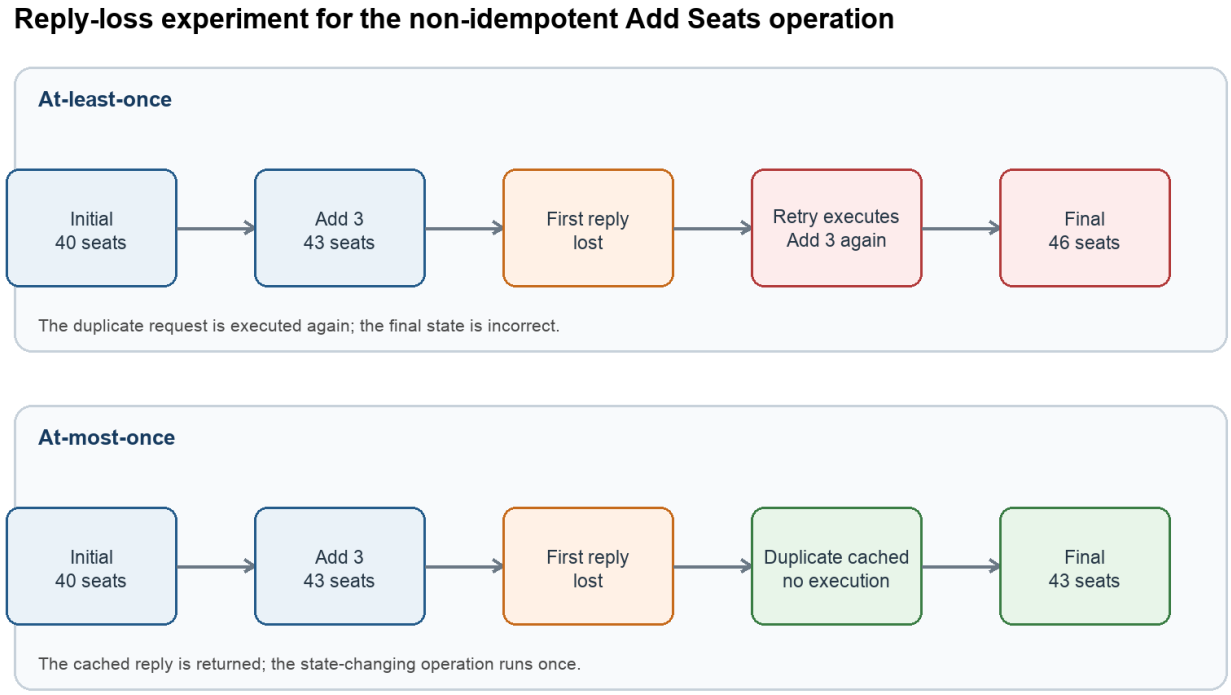


Figure 2 Effect of a lost reply on a non-idempotent operation

6.4 Loss simulation

The server supports probabilistic request and reply loss rates from zero to one, plus deterministic options that drop the first request or first reply for each request ID. Deterministic first-reply loss is used for the

report experiment because it is repeatable: the server executes the operation, withholds the reply, and forces exactly the retry scenario needed to compare invocation semantics.

7 Implementation Details

7.1 C++17 server

The server uses POSIX IPv4 UDP sockets and a receive loop with a short socket timeout. Flight records are stored in an unordered map keyed by flight ID. Monitor records use steady-clock expiry times so wall-clock changes cannot extend or shorten a registration. At-most-once history uses an unordered map for lookup and a deque for bounded eviction. A Mersenne Twister engine supports reproducible probabilistic loss when a random seed is supplied.

BinaryWriter and BinaryReader construct and parse bytes explicitly. They never transmit a C++ structure directly, avoiding padding and host-endian dependence. Double values are converted to their 64-bit IEEE-754 bit patterns with memcpy and then emitted most-significant byte first. The server compiles with C++17, optimization, and the Wall, Wextra, and Wpedantic warning sets.

7.2 Python client

ReliableFlightClient centralizes request construction, retry behavior, sender verification, reply matching, status handling, and payload decoding. The console layer calls typed convenience methods for each operation and reports RemoteError, TimeoutError, validation errors, and protocol errors without terminating the menu. During monitoring, the client temporarily changes the socket timeout to the remaining interval and accepts only valid monitoring callbacks from the configured server.

7.3 Build and source organization

Path	Purpose
src/server.cpp	Complete C++17 server, protocol codec, services, callbacks, histories, and loss simulation
src/client.py	Python client library and interactive console interface
src/common/message.py	Python protocol constants, message framing, and operation payload codecs
src/common/flight.py	Python flight model and flight-record decoding
tests/test_system.py	Cross-language protocol and integration tests
Makefile	Reproducible C++ compilation and test entry points

8 Experiments and Validation

8.1 Test environment and method

The final automated validation was performed on macOS arm64 using Apple Clang 21.0.0 and Python 3.14.7. Each integration test reserves a local UDP port, starts the compiled C++ server as a separate process, and sends requests through the production Python client. The tests therefore validate the same cross-language executable and codecs used for the demonstration rather than a mock server.

8.2 Automated results

Test	Observed result	Status
Flight binary round trip with variable-length Unicode strings	Original and decoded records match	PASS
Malformed payload length	Decoder raises ProtocolError	PASS
All six Python-to-C++ service calls and error paths	Expected values and statuses returned	PASS
C++ callback to Python monitoring client	Flight 1001 callback reports 39 seats after one reservation	PASS
At-least-once with first reply lost	Add 3 changes 40 to 46	PASS
At-most-once with first reply lost	Add 3 changes 40 to 43	PASS

The test suite completed six tests with no failures or errors.

8.3 Invocation-semantics experiment

Mode	Initial	First execution	Retry handling	Final	Correct
At-least-once	40	43; reply lost	Operation executes again	46	No
At-most-once	40	43; reply lost	Cached reply returned	43	Yes

The experiment confirms the expected theory. Retransmission alone provides eventual execution but cannot distinguish a new request from a duplicate. The request identifier and server-side history are what make at-most-once safe for non-idempotent operations. The history must be updated before the reply is sent; otherwise a reply lost between execution and history insertion would still allow re-execution.

8.4 Requirements traceability

Project requirement	Implementation evidence
UDP client and server	Python and C++ use IPv4 SOCK_DGRAM sockets
Manual marshalling	Independent BinaryWriter and BinaryReader implementations; no serialization framework
Four required services	Operation codes 1 to 4 and integration coverage
Two additional operations	Set Price is idempotent; Add Seats is non-idempotent
Callback monitoring	Monitor registry plus C++-to-Python callback test
At-least-once and at-most-once	Selectable server modes, client timeout/retry, and reply history
Loss simulation	Deterministic first loss and configurable probabilistic rates
Different implementation languages	C++17 server interoperates with Python 3 client

9 Deployment and Demonstration

9.1 Build and start the server

```
make
./flight_server --host 0.0.0.0 --port 8888 --semantics at-most-once
```

The server computer must allow inbound UDP traffic on port 8888. The host value 0.0.0.0 listens on all local IPv4 interfaces. The server's LAN address, such as 192.168.1.20, is then supplied to each client.

9.2 Start clients on separate computers

```
python3 src/client.py --host 192.168.1.20 --port 8888 --semantics at-most-once
```

The required callback demonstration uses three computers on the same network: one C++ server computer, one Python client monitoring flight 1001, and another Python client reserving seats. The monitoring client selects operation 4 and a 30-second interval. During that interval, the reservation client selects operation 3. The server logs both requests and the callback destination, and the monitoring client prints the new availability.

9.3 Repeatable semantics demonstration

```
# At-least-once
./flight_server --semantics at-least-once --drop-first-reply
python3 src/client.py --semantics at-least-once --timeout 0.5

# At-most-once
./flight_server --semantics at-most-once --drop-first-reply
python3 src/client.py --semantics at-most-once --timeout 0.5
```

For each mode, the server is restarted to reset flight 1001 to 40 seats. Operation 6 adds three seats. The at-least-once run reports 46 after retry, while the at-most-once run reports 43 and logs that the duplicate received a cached reply.

10 Limitations and Conclusion

10.1 Limitations

- Flight data, monitor registrations, and duplicate history are held in memory and are lost when the server restarts, which is permitted by the project brief.
- The at-most-once history is bounded. A duplicate arriving after its entry has been evicted may execute again; a production system would use durable identities and storage.
- The protocol keeps each message within one UDP datagram and does not implement fragmentation, congestion control, authentication, or encryption.
- The server is single-threaded according to the permitted request-spacing assumption. A high-load production server would require explicit synchronization or an event-driven scalability design.
- The demonstration depends on a network that permits peer-to-peer UDP traffic; some campus Wi-Fi networks isolate clients, so a private hotspot may be preferable.

10.2 Conclusion

The completed system satisfies the required flight services, manual marshalling, UDP transport, callback monitoring, selectable invocation semantics, and message-loss simulation. It also fulfills the optional cross-language objective through a C++17 server and Python client that share only a documented binary wire format. The reply-loss experiment demonstrates why at-most-once execution is necessary for non-idempotent operations, while the automated integration tests provide evidence that the two independent language implementations interoperate correctly.

References

[1] SC6103 Distributed Systems, Design and Implementation of a Distributed Flight Information System, Course Project for 2026-2027 Trimester 1.

Appendix A Protocol Value Summary

Category	Value	Meaning
Kind	1 / 2 / 3	Request / reply / callback
Operation	1	Search Flights
Operation	2	Get Flight
Operation	3	Reserve Seats
Operation	4	Monitor Seats
Operation	5	Set Price
Operation	6	Add Seats
Status	0	OK
Status	1	BAD_REQUEST
Status	2	NOT_FOUND
Status	3	INSUFFICIENT_SEATS
Status	4	INTERNAL_ERROR

Appendix B Test Command

```
make test
```

The Makefile compiles src/server.cpp into flight_server and invokes Python unittest discovery. The tests start and stop isolated C++ server processes on temporary localhost UDP ports.